

Joint Chinese Word Segmentation and Part-of-Speech Tagging with the Transformer

Paul He
ETH Zürich
paulhe1@ethz.ch

Abstract

Chinese text analysis presents unique challenges, as words in Chinese sentences are not demarcated, making tasks like part-of-speech (POS) tagging and Chinese word segmentation (CWS) essential preliminary steps. This paper explores the effectiveness using the Transformer architecture to perform joint CWS and POS tagging, where BERT embedding are used to encode contextual information. We were able to achieve strong performances with significantly fewer training resources.¹

1 Introduction

Chinese word segmentation (CWS) involves of dividing a Chinese sentence into distinct groups of characters that form meaningful units. Part-of-speech tagging (POS) is a labelling task that assigns a POS tag on each segmented word. Past works have shown that performing CWS and POS tagging jointly were more effective than sequentially, and that in Chinese, character based POS assignments were more effective than word based.(Ng and Low, 2004). Many papers explored performing the task as a character-based sequence labeling problem. All at once, character-based POS tagger and segmenter resulted the best performance in comparison to word-based and one-at-a-time models (Ng and Low, 2004).

In this paper, we propose to perform character based CWS and POS tagging using the transformer architecture proposed by Vaswani et al. 2017, where we use BERT embedding to encode contextual information (Devlin et al., 2019). Our model achieves competitive performances to state-of-the-art models on the Chinese Penn Treebank (CTB) with fewer training resources (Nianwen Xue, 2010, Nianwen Xue, 2016).

2 Background

Many neural models in the past decade effectively reduced the efforts in feature engineering (Shao et al., 2017, Zhang et al., 2018). However, contextual features need to be appropriately handled. Consider the example “街道路面积水” (water accumulation on street surface) can easily be incorrectly tagged where “路面”(street surface) and “面积” (area) are both n-gram features of “面” which could mean noodle or surface. “街道” (street) and “道路”(road) are both n-gram features of “道” (road). All extra contextual information is treated uniformly without considering how important each piece is to the current task. Frameworks such as recurrent neural networks (RNN), generation is based only on the most recent output of the model (Vaswani et al., 2017). The information is limited to a fixed size vector which will be harder to compress contextual information for longer sequences of input into such information, and the data relevant at different time steps may also change (Vaswani et al., 2017). There have been studies that use BERT encoders, with extra embedding information and a CRF decoder to improve joint CWS and POS tagging(Tian et al., 2020a).

3 Model

In this section, we describe some background behind the work and the structure of our model.

3.1 Tagging

Let $T = \{B, M, E, S\}$ be the set of boundary tags and P be the set of POS tags. Each character is assigned a target value by concatenating an element from T and another from P ². The set of classes can be defined as $\mathcal{Y} = \{t \circ p | t \in T, p \in P\}$. Figure 1 shows an example of a POS tagged and segmented sentence.

¹ <https://github.com/Hepaul17/SegPosCN>

²See appendix for POS tags

Figure 1: Example of Joint POS Tagged and Segmented Sentence

3.2 Embedding Layer

For the inputs, we use `bert-base-chinese` to get their embedding. These embedding contain contextual information and will be in the dimension of $d_{\text{model}} = 768$. We will be fine-tuning the `bert-base-chinese` model. For the tags, we will train our own embedding, converting the tag (output tokens) to $d_{\text{model}} = 768$.

In addition, we will concatenate the tags and `bert-base-chinese` embedding for the characters, then have a linear projection layer that projects the concatenated embedding back to dimension d_{model} . This allows the decoder to know which tag it is decoding for. Since we are using `bert-base-chinese` for the inputs, we do not need to train our own encoder, which significantly speeds up training.

3.3 The Attention Mechanism

The Attention Mechanism allows a model to focus different parts of the input sentence when predicting each part of an output sequence (Vaswani et al., 2017). This enables long-range dependencies to be captured more effectively (Vaswani et al., 2017). In our model, the key $\mathbf{k}_i = \mathbf{v}_i = \mathbf{h}_i^{(e)}$ is the hidden state produced by the BERT embedding at position i , the query $\mathbf{q}_t = \mathbf{h}_t^{(d)}$ is the hidden state produced by the decoder at position t , the context vector $\mathbf{c}_t = \alpha \mathbf{V}$, and the vertically stacked hidden states $\mathbf{K} = \mathbf{V} = \mathbf{H}^{(e)}$ and $\mathbf{Q} = \mathbf{H}^{(d)}$. The dimensions of the queries and keys are d_k and the dimensions of the value is d_v . We can compute the attention weights as

$$\alpha = \text{softmax}(\text{score}(\mathbf{q}, \mathbf{K})) \quad (1)$$

3.3.1 Scaled Dot-Product Attention

We will use the Scaled Dot-Product Attention, which has $\alpha = \text{score}(\mathbf{q}_t, \mathbf{K}_i) = \frac{\mathbf{q}_t \mathbf{K}_i^\top}{\sqrt{d_k}}$. The attention function is then

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \quad (2)$$

where the softmax is applied column wise. The scale $\frac{1}{\sqrt{d_k}}$ was proposed to prevent the softmax

from pushing into extremely small gradients, which is caused by the large dot product (Vaswani et al., 2017).

3.3.2 Multi-head Attention

Let $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ where h is the number of heads. Then, we can define each head as:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \quad (3)$$

By concatenating each of these heads, we get the Multi-head Attention (Vaswani et al., 2017):

$$\text{MHSA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1 \dots \text{head}_h) \mathbf{W}^O \quad (4)$$

In our model, we use $h = 8$, we let $d_k = d_v = d_{\text{model}}/h = 96$, since we are using $d_{\text{model}} = 768$

3.3.3 Positional Encoding

Unlike RNNs and CNNs, transformers contain no recurrence or convolution (Vaswani et al., 2017). The attention mechanism above is position equivariant, that is, given a permutation matrix $\Pi \in \mathbb{R}^{N \times N}$, with any input $\mathbf{Z} \in \mathbb{R}^{N \times N}$, we have:

$$\text{Attention}(\Pi \mathbf{Z}) = \Pi \text{Attention}(\mathbf{Z}) \quad (5)$$

This means, we need to give positional information to our model, otherwise, “我吃了奶酪” (I ate cheese) and “奶酪吃了我” (cheese ate me) will be interpreted the same to our model, though, they have very distinct meanings despite having the same characters. `bert-base-chinese` already contains positional information in their embedding, we still need to add it to our output embedding. We will use the same absolute positional encodings as the original Transformer paper (Vaswani et al., 2017), for even indices we have:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{1000^{2i/d_{\text{model}}}}\right) \quad (6)$$

and for odd indices we have:

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{1000^{2i/d_{\text{model}}}}\right) \quad (7)$$

3.4 Masking

The inputs, outputs masks have a 1 for non PAD indices and 0 otherwise. Since we also give the outputs during training, we need to correctly apply subsequent masks along with shifting output embedding to the right to ensure the auto-regressive

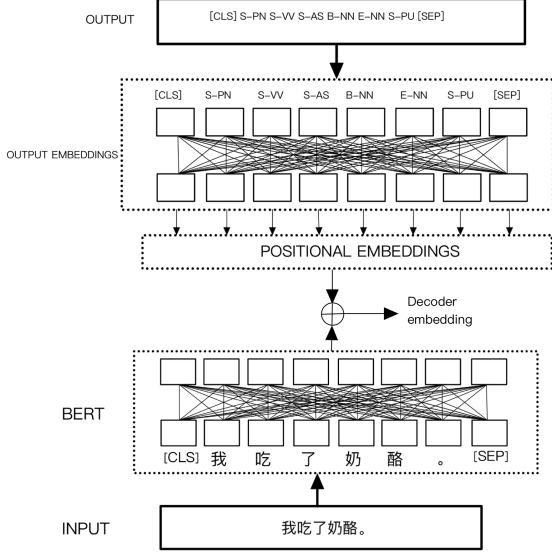


Figure 2: Embedding for the Decoder, obtained by adding input embedding, once decoder embedding obtained, need to project to d_{model}

nature of the model (Vaswani et al., 2017). The subsequent mask is a lower triangular matrix filled with 1s (Vaswani et al., 2017). The masking for the output is done as taking the operation with the output mask.

3.5 Decoding layer

In our model, we use $N = 3$ layers. Each layer consists of three sublayers: multi-head attention layer, another multi-head attention layer over the output of bert-base-chinese, and a position-wise fully connected feed-forward network (Vaswani et al., 2017). Each layer is connected via residual connections, followed by layer normalization (Vaswani et al., 2017).

3.6 Beam Search

During inference, we want to find a tagging $\mathbf{y}$ with high probability for our input sentence $\mathbf{x}$. Dynamic programming techniques would be as slow as brute-force methods. Beam search is similar to the idea of breadth-first-search, except its width is limited to the size of $k \in \mathbb{Z}_+$, meaning a maximum of k hypothesis are expanded at each time step (Meister et al., 2020). We will perform beam search over the tag space, with beam size $k = 4$. At each position in the sequence, the softmax function generates a probability distribution over all possible output tags, and these probabilities sum up to 1 (Meister

et al., 2020). This happens independently at each position in the sequence, we can say our model locally normalized. With weight θ and time t , our model is defined as p_θ

$$p_\theta(\mathbf{y}|\mathbf{x}) = \prod_{t=1}^{|\mathbf{y}|} p_\theta(y_t|\mathbf{x}, \mathbf{y}_{<t}) \quad (8)$$

where $p_\theta(\cdot|\mathbf{x})$ is a distribution with support over $\mathcal{Y}$. When searching over the tag space, we can express the task as a recursion:

$$Y_0 = \{\text{BOS}\} \quad (9)$$

$$Y_t = \arg \max_{Y' \subseteq \mathcal{B}_t | |Y'|=k} p_\theta(Y'|\mathbf{x}) \quad (10)$$

where the candidate set $\mathcal{B}_t$ with $t > 0$ is defined as:

$$\mathcal{B}_t = \{\mathbf{y}_{t-1} \circ y | y \in \mathcal{Y}\} \quad (11)$$

Normally, $\circ$ denotes the concatenation of two strings, but since we are searching over the tags, we are just simply appending the tags to the previous tags.

4 Experiments

4.1 Training

We trained on the Chinese Penn Treebank (CTB7 and CTB9). The CTB7 and CTB9 datasets contain 26476 and 66884 training sentences respectively. We followed previous studies on how to obtain the training, development and testing datasets (Tian et al., 2020b). The sentences are shuffled randomly into batches with a size of 128 and 12 epochs. We trained our models on an Apple M1 Pro CPU. Training the CTB7 took approximately 4 hours, while the CTB9 took approximately 9 hours.

4.2 Optimizer

As the original transformer paper achieved excellent results, we will follow a similar approach by using the Adam Optimizer with $\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$ (Kingma and Ba, 2017). The learning rate is varied over the duration of training as:

$$\alpha = d_{\text{model}}^{-0.5} \min\{n_{\text{step}}^{-0.5}, n_{\text{step}} \times n_{\text{warmup}}^{-1.5}\} \quad (12)$$

where $n_{\text{warmup}} = 250$. We also applied dropout to the output of every sublayer, we use a dropout rate of $P_{\text{drop}} = 0.1$ (Hinton et al., 2012).

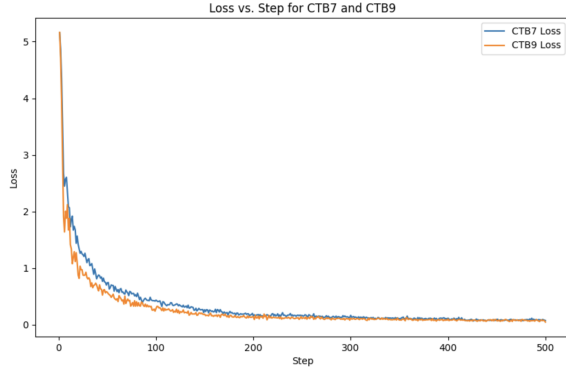


Figure 3: Training loss for the first 500 steps (corresponds 1-2 epochs depending on size of training set with batch size 128).

	Joint	CWS	POS
CTB7	89.88%	94.24%	91.56%
CTB9	91.99%	96.29%	93.37%

Table 1: Validation Accuracy for CTB7 and CTB9

4.3 Results

As with previous studies (Tian et al., 2020b), we compared the performance of our model on the development set. Table 1 shows the validation accuracy of CTB7 and CTB9 on a set of 8531 and 9431 sentences respectively. We compared segmentation performance, POS tagging performance on correctly segmented words, and the joint performance of segmentation and POS tagging. Since our model performs well on CWS, followed by POS tagging given a correctly segmented character, and then the joint task, this leaves room for further exploration on how transformers deal with CWS and POS tagging sequentially. Figure 3 shows the training loss for the first 500 steps. The loss decreases significantly in the first 100 steps. Around 6000 steps, the loss is around 0.007. This indicates our model learns fast, capturing most patterns within just one epoch of training.

4.4 Comparisons and Analysis

Table 3, 4 shows the performance of our model compared to other studies on CTB7 and CTB9. In terms of CWS, our model delivers similar per-

	Joint	CWS	POS
CTB7	90.09%	95.37%	93.95%
CTB9	90.72%	95.77%	94.01%

Table 2: Validation F-scores for CTB7 and CTB9

	CWS	Joint
Our Model	95.37%	90.09%
Wang et al., 2011	95.36%	90.27%
Zhang et al., 2018	96.23%	91.25%
Kurita et al., 2017	96.23%	91.25%
Tian et al., 2020b	97.16%	94.09%

Table 3: CTB7 CWS and Joint F scores Compared to other models

	CWS	Joint
Our Model	95.77%	90.72%
Shao et al., 2017	96.67%	92.34%
Tian et al., 2020b	97.67%	94.87%

Table 4: CTB9 CWS and Joint F scores Compared to other models

formance to the Multi-Criteria CWS model with Transformer encoder, CRF decoder but with 10 times less epochs proposed by (Qiu et al., 2020). The model proposed by Tian et al. 2020b, where they used a BERT encoder, but with multi-channel attention module with n-grams and a CRF decoder had the best performance among recent models. A possible reason for this is that they have injected extra information in addition to the BERT encodings via multi-channel attention module with n-grams for the inputs (Tian et al., 2020b), their general structure is similar to Qiu et al. 2020, except they only used it for CWS, which indicates their extra information significantly improved their results. CTB9 has about triple the amount of data compared to CTB7, yet their F scores are still quite close in terms of segmentation, indicating our model scales well with the size of dataset. This matches what Figure 3 shows, as our model learns most features within 100 step. It is also worth noting the slight increase in the F score for CTB7 when we only consider texts length ≥ 40 , indicating a potential limitation of our model with shorter texts or phrases without enough context Tian et al. 2020b mentioned that most Chinese words contain 2 characters.

It is worth noting that our model had consistent

	Joint	CWS	POS
CTB7	90.37%	95.67%	94.23%
CTB9	89.42%	95.56%	93.47%

Table 5: F scores for text with length ≥ 40

performance on texts with more than 40 characters. For CTB7, our statistics show that 30% of the dataset contained texts longer than 40 characters, while in CTB9, less than 20% occurred. Table 5 shows that our model across the datasets have consistent performances.

5 Conclusion

In this work, we explored using the full Transformer architecture on join Chinese word segmentation and part-of-speech tagging, with the outputs embedded with the input BERT embedding.

Although having a slightly lower performance than existing state-of-the-art models, our Transformer models are trained significantly faster than recurrent architectures. Our model was also able to correctly segment ambiguous sentences, and capture long range dependencies, which show how powerful the attention mechanism is. We are interested in improving our model in the future by providing more contextual information for the input embedding.

References

- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. 2012. [Improving neural networks by preventing co-adaptation of feature detectors](#).
- Diederik P. Kingma and Jimmy Ba. 2017. [Adam: A method for stochastic optimization](#).
- Shuhei Kurita, Daisuke Kawahara, and Sadao Kurohashi. 2017. [Neural joint model for transition-based Chinese syntactic analysis](#). In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1204–1214, Vancouver, Canada. Association for Computational Linguistics.
- Clara Meister, Ryan Cotterell, and Tim Vieira. 2020. [If beam search is the answer, what was the question?](#) In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2173–2185, Online. Association for Computational Linguistics.
- Hwee Tou Ng and Jin Kiat Low. 2004. Chinese part-of-speech tagging: One-at-a-time or all-at-once? word-based or character-based. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, page 277–284, Barcelona, Spain.
- Xiuhong Zhong Martha Palmer Fei Xia Fu-Dong Chiou Meiyu Chang Nianwen Xue, Zixin Jiang. 2010. [Chinese treebank 7.0](#).
- Zixin Jiang Martha Palmer Fei Xia Fu-Dong Chiou Meiyu Chang Nianwen Xue, Xiuhong Zhang. 2016. [Chinese treebank 9.0](#).
- Xipeng Qiu, Hengzhi Pei, Hang Yan, and Xuanjing Huang. 2020. [A concise model for multi-criteria Chinese word segmentation with transformer encoder](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2887–2897, Online. Association for Computational Linguistics.
- Yan Shao, Christian Hardmeier, Jörg Tiedemann, and Joakim Nivre. 2017. [Character-based joint segmentation and POS tagging for Chinese using bidirectional RNN-CRF](#). In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 173–183, Taipei, Taiwan. Asian Federation of Natural Language Processing.
- Yuanhe Tian, Yan Song, and Fei Xia. 2020a. [Joint chinese word segmentation and part-of-speech tagging via multi-channel attention of character n-grams](#). In *Proceedings of the 28th International Conference on Computational Linguistics*, page 2073–2084, Barcelona, Spain (Online). International Committee on Computational Linguistics.
- Yuanhe Tian, Yan Song, and Fei Xia. 2020b. [Joint Chinese word segmentation and part-of-speech tagging via multi-channel attention of character n-grams](#). In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 2073–2084, Barcelona, Spain (Online). International Committee on Computational Linguistics.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#).
- Yiou Wang, Jun’ichi Kazama, Yoshimasa Tsuruoka, Wenliang Chen, Yujie Zhang, and Kentaro Torisawa. 2011. [Improving Chinese word segmentation and POS tagging with semi-supervised methods using large auto-analyzed data](#). In *Proceedings of 5th International Joint Conference on Natural Language Processing*, pages 309–317, Chiang Mai, Thailand. Asian Federation of Natural Language Processing.
- Meishan Zhang, Nan Yu, and Guohong Fu. 2018. [A simple and effective neural model for joint word segmentation and pos tagging](#). *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 26(9):1528–1538.

Appendix A: All POS Tags

The dataset contains of 33 POS tags:

- VA: Predicative adjective
- VC: Copula (like the English "be" in certain contexts)
- VE: Existential verb (similar to the English "have" in contexts like "There is")
- VV: Other verbs
- NR: Proper noun
- NT: Temporal noun (referring to time-related concepts)
- NN: Common noun
- LC: Localizer (indicating a spatial or temporal location)
- PN: Pronoun
- DT: Determiner
- CD: Cardinal number
- OD: Ordinal number
- M: Measure word (quantifiers and classifiers)
- AD: Adverb
- P: Preposition
- CC: Coordinating conjunction
- CS: Subordinating conjunction
- DEC: A particle used after a verb to form a relative clause (similar to "that" in English)
- DEG: Associative particle (equivalent to the English "'s" or "of")
- DER: A particle used after a verb or an adjective in certain cleft constructions
- DEV: A particle used to form adverbial expressions
- SP: Sentence-final particle
- AS: Aspect particle (indicating aspects like completed action)
- ETC: Etcetera (used in lists to indicate "and so on")
- MSP: Other particles
- IJ: Interjection
- ON: Onomatopoeia
- PU: Punctuation
- JJ: Other noun-modifier (adjective)
- FW: Foreign words (words borrowed from other languages)
- LB: Long Bei construction marker (a special structure in Chinese grammar)
- SB: Short Bei construction marker
- BA: Ba construction marker (another unique structure in Chinese grammar)